🧩 Problemas

Tabla de contenidos

1 Registro de llamadas 1
1.1 Requisitos 2
1.2 Ayuda 2
2 Fibonacci el memorioso 2
2.1 Ayuda 2

1 Registro de llamadas

En programación, el *logging* es el proceso de registrar eventos durante la ejecución de un programa. Estos registros, llamados *logs*, permiten mantener un historial de lo que ocurrió, incluyendo estados del sistema, acciones realizadas y posibles errores o advertencias.

En este problema se propone implementar un decorador llamado `log` que registre, en un archivo, los valores de entrada y salida de cada llamada a las funciones decoradas. El decorador debe recibir como argumento la ruta del archivo donde se guardarán los mensajes.

Por ejemplo, el siguiente código:

```
import time

@log("registro.log")
def f(x, y):
    return x + y

@log("registro.log")
def g(a, b):
    return a + b ** 2

f(3, 9)
time.sleep(1)
f(256, 256)
time.sleep(3)
g(3, 2)
f(7, 8)
```

Debería generar un archivo `registro.log` con un contenido similar al siguiente:

```
2025-09-02 18:04:07 - | f(x=3, y=9) -> 12
2025-09-02 18:04:08 - | f(x=256, y=256) -> 512
2025-09-02 18:04:11 - | g(a=3, b=2) -> 7
2025-09-02 18:04:11 - | f(x=7, y=8) -> 15
```

1.1 Requisitos

- Si el archivo no existe, debe crearse automáticamente.
- Si el archivo ya existe, los nuevos registros deben agregarse al final, sin sobrescribir los anteriores.

1.2 Ayuda

- Para obtener el momento exacto de la llamada, puede usar `datetime.now()` del módulo `datetime`.
- Para formatear ese valor como texto, utilice el método `.strftime`.

Por ejemplo:

```
datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```
2025-09-02 18:30:39
```

2 Fibonacci el memorioso

La implementación recursiva más directa de la secuencia de Fibonacci es tan concisa como ineficiente.

```
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)
```

Al invocar `fibonacci(7)`, se realizan llamadas a `fibonacci(6)` y `fibonacci(5)`. Luego, `fibonacci(6)` vuelve a llamar a `fibonacci(5)` y `fibonacci(4)`, y así sucesivamente. Muchas de estas llamadas se repiten: por ejemplo, `fibonacci(5)` se calcula más de una vez.

Este comportamiento muestra que los subproblemas se superponen: los mismos valores se recalculan de manera redundante.

El objetivo de este ejercicio es diseñar una función capaz de recordar resultados ya obtenidos, evitando recomputarlos cada vez que aparecen. Para lograrlo, podemos construir una función recursiva que memorice sus valores anteriores y, así, avance en la secuencia sin rehacer todos los cálculos previos.

2.1 Ayuda

Considere una *function factory* que devuelva la función recursiva. Dentro del ámbito de la *function factory* puede existir una estructura de datos mutable que almacene los resultados ya calculados.